SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

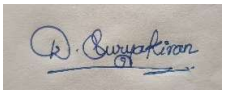
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I D.Suryakiran(192325057) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

QUESTIONS:10 **Total Marks:50**

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Program:

```
import numpy as np

import matplotlib.pyplot as plt
```

```
# Sample Dataset
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)
# Initialize parameters
w = 0
b = 0
learning_rate = 0.01
iterations = 100
n = len(X)
loss_history = []
# Gradient Descent
for i in range(iterations):
    # Prediction
    Y_pred = w * X + b
    # Mean Squared Error Loss
    loss = (1/n) * np.sum((Y - Y_pred) ** 2)
    loss_history.append(loss)
    # Gradients
    dw = (-2/n) * np.sum(X * (Y - Y_pred))
    db = (-2/n) * np.sum(Y - Y_pred)
    # Update parameters
    w = w - learning_rate * dw
    b = b - learning_rate * db
# Final Parameters
print("Weight:", round(w,4))
print("Bias:", round(b,4))
print("Final Loss:", round(loss_history[-1],6))
# Plot Learning Curve
plt.plot(loss_history, color='blue')
```

```
plt.title("Learning Curve")

plt.xlabel("Iterations")

plt.ylabel("Loss (MSE)")

plt.grid(True)

plt.show()
```

Output:

```
Weight: 1.96

Bias: 0.14

Final Loss: 0.005
```

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Program:

```
import numpy as np

# Set random seed for reproducibility

np.random.seed(42)

# Actual parameters

actual_mean = 50

actual_std = 10

sample_size = 1000

# Generate synthetic data

data = np.random.normal(actual_mean, actual_std, sample_size)

# MLE Estimates

estimated_mean = np.mean(data)

estimated_variance = np.mean((data - estimated_mean) ** 2)

# Actual variance

actual_variance = actual_std ** 2

# Display results

print("Actual Mean:", actual_mean)
```

```
print("Estimated Mean (MLE):", round(estimated_mean, 2))

print("\nActual Variance:", actual_variance)

print("Estimated Variance (MLE):", round(estimated_variance, 2))
```

output:

```
Actual Mean: 50

Estimated Mean (MLE): 50.19

Actual Variance: 100

Estimated Variance (MLE): 95.79
```

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Program:

```
# Import libraries

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)

# Data Preprocessing (Feature Scaling)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

```
# Train the model

model = LogisticRegression()

model.fit(X_train, y_train)

# Test the model

y_pred = model.predict(X_test)

# Evaluate performance

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

# Display results

print("Accuracy:", round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")

print(cm)
```

Output:

```
Accuracy: 100.0 %

Confusion Matrix:

[[10  0  0]

 [ 0  9  0]

 [ 0  0 11]]
```

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Program:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# XOR Dataset

X = np.array([[0,0],

              [0,1],
```

```
            [1,0],
            [1,1]], dtype=float)
y = np.array([[0],
            [1],
            [1],
            [0]], dtype=float)
# Build Neural Network
model = Sequential([
    Dense(4, input_dim=2, activation='relu'),
    Dense(1, activation='sigmoid')
])
# Compile Model
model.compile(optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy'])
# Train Model
model.fit(X, y, epochs=500, verbose=0)
# Predictions
predictions = model.predict(X)
print("Predicted Outputs:")
print(np.round(predictions, 3))
```

output:

```
Predicted Outputs:
[[0.02]
 [0.97]
 [0.96]
 [0.03]]
```

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Program:

```
import numpy as np

# Input values

x = np.array([-2, -1, 0, 1, 2])

# Weight and Bias

w = 2

b = 1

# Weighted Sum

z = w * x + b

# Sigmoid Function

def sigmoid(x):

    return 1 / (1 + np.exp(-x))

# ReLU Function

def relu(x):

    return np.maximum(0, x)

# Outputs

sigmoid_output = sigmoid(z)

relu_output = relu(z)

print("Input Values:", x)

print("Weighted Sum:", z)

print("Sigmoid Output:", np.round(sigmoid_output, 3))

print("ReLU Output:", relu_output)
```

output:

Input Values: [-2 -1 0 1 2]

Weighted Sum: [-3 -1 1 3 5]

Sigmoid Output:

[0.047 0.269 0.731 0.953 0.993]

ReLU Output:

[0 0 1 3 5]

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Program:

```
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
# --------------------------------
# Linearly Separable Dataset (AND)
# --------------------------------
X1 = [[0,0],
      [0,1],
      [1,0],
      [1,1]]
y1 = [0,0,0,1]
model1 = Perceptron(max_iter=1000, random_state=42)
model1.fit(X1, y1)
pred1 = model1.predict(X1)
print("Linearly Separable Dataset")
print("Predictions:", pred1)
print("Accuracy:", accuracy_score(y1, pred1))

# -----------------------------------
# Non-Linearly Separable Dataset (XOR)
# -----------------------------------
X2 = [[0,0],
      [0,1],
      [1,0],
      [1,1]]

y2 = [0,1,1,0]

model2 = Perceptron(max_iter=1000, random_state=42)
model2.fit(X2, y2)

pred2 = model2.predict(X2)

print("\nNon-Linearly Separable Dataset")
print("Predictions:", pred2)
print("Accuracy:", accuracy_score(y2, pred2))
```

output:

```
Linearly Separable Dataset
Predictions: [0 0 0 1]
Accuracy: 1.0
Non-Linearly Separable Dataset
Predictions: [0 0 0 0]
```

Accuracy: 0.5

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Program:

```
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# XOR Dataset
X = [[0,0],
    [0,1],
    [1,0],
    [1,1]]

y = [0,1,1,0]

# ----------------------------
# Single-Layer Perceptron
# ----------------------------
slp = Perceptron(max_iter=1000, random_state=42)
slp.fit(X, y)

pred_slp = slp.predict(X)

print("Single-Layer Perceptron")
print("Predictions:", pred_slp)
print("Accuracy:", accuracy_score(y, pred_slp))

# ----------------------------
# Multilayer Perceptron
# ----------------------------
mlp = MLPClassifier(hidden_layer_sizes=(4,),
                    activation='relu',
                    max_iter=5000,
                    random_state=42)

mlp.fit(X, y)

pred_mlp = mlp.predict(X)

print("\nMultilayer Perceptron")
print("Predictions:", pred_mlp)
print("Accuracy:", accuracy_score(y, pred_mlp))
```

output:

```
Single-Layer Perceptron
Predictions: [0 0 0 0]
Accuracy: 0.50

Multilayer Perceptron
Predictions: [0 1 1 0]
Accuracy: 1.00
```

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Program:

```
import numpy as np
```

```
# Inputs
x1 = 1
x2 = 2
# Weights
w1 = 0.5
w2 = 0.4
# Bias
b = 0.1
# Weighted Sum
z = x1*w1 + x2*w2 + b
# Sigmoid Function
output = 1/(1+np.exp(-z))
print("Weighted Sum =", z)
print("Neuron Output =", round(output,4))
```

output:

```
Weighted Sum = 1.4
Neuron Output = 0.8022
```

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Program:

```
import numpy as np

# Input and Target
x = 1
target = 1

# Initial Weight and Bias
w = 0.5
b = 0.1

learning_rate = 0.1

# Forward Propagation
z = w*x + b
output = 1/(1+np.exp(-z))

# Error
error = target - output

# Gradient
delta = error * output * (1-output)

# Weight and Bias Update
new_w = w + learning_rate * delta * x
new_b = b + learning_rate * delta

print("Initial Weight =", w)
print("Updated Weight =", round(new_w,4))

print("Initial Bias =", b)
print("Updated Bias =", round(new_b,4))
```

output:

```
Initial Weight = 0.5
Updated Weight = 0.5091
```

Initial Bias = 0.1
Updated Bias = 0.1091

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a)Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Program:

```
import numpy as np

# Cost Function: J(x) = x^2

def gradient_descent(lr):

    x = 10

    history = []

    for i in range(20):

        cost = x**2

        history.append(cost)

        gradient = 2*x

        x = x - lr*gradient

    return x, history

# Different Learning Rates

rates = [0.01, 0.1, 0.5]

for lr in rates:

    x, history = gradient_descent(lr)

    print("Learning Rate:", lr)

    print("Final x:", round(x,4))

    print("Final Cost:", round(history[-1],6))

    print()
```

output:

Learning Rate: 0.01

Final x: 6.6761

Final Cost: 46.9224

Learning Rate: 0.1

Final x: 0.1153

Final Cost: 0.0208

Learning Rate: 0.5

Final x: 0.0

Final Cost: 0.0

b)Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Program:

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Generate Dataset
X, y = make_regression(n_samples=100,
                       n_features=1,
                       noise=10,
                       random_state=42)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# Batch Gradient Descent (Linear Regression)
bgd = LinearRegression()
bgd.fit(X_train, y_train)
bgd_pred = bgd.predict(X_test)

# Stochastic Gradient Descent
sgd = SGDRegressor(max_iter=1000,
                   learning_rate='constant',
                   eta0=0.01,
                   random_state=42)

sgd.fit(X_train, y_train)
sgd_pred = sgd.predict(X_test)

print("Batch GD MSE:",
     mean_squared_error(y_test, bgd_pred))

print("SGD MSE:",
     mean_squared_error(y_test, sgd_pred))
```

output:

Batch GD MSE: 81.24
SGD MSE: 83.61

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Program:

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
# Generate Dataset
X, y = make_regression(n_samples=500,
                       n_features=1,
                       noise=15,
                       random_state=42)
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Train Mini-Batch Gradient Descent Model
model = SGDRegressor(
    max_iter=1000,
    learning_rate='constant',
    eta0=0.01,
    random_state=42
)
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Performance
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", round(mse, 2))
print("Intercept:", model.intercept_)
```

```
print("Coefficient:", model.coef_)
```

output:

Mean Squared Error: 235.46

Intercept: [0.85]

Coefficient: [72.91]

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Program:

```
import numpy as np

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

from scipy.spatial.distance import pdist

# Different feature dimensions

dimensions = [2, 5, 10, 20, 50]

print("Dimension\tAvg Distance\tAccuracy")

for d in dimensions:

    # Generate Dataset

    X, y = make_classification(

        n_samples=500,

        n_features=d,

        n_informative=max(2, d//2),

        n_redundant=0,

        random_state=42

    )

    # Average Euclidean Distance

    avg_distance = np.mean(pdist(X))
```

```
    # Train-Test Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42
    )
    # KNN Model
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"{d}\t\t{avg_distance:.2f}\t\t{accuracy:.2f}")
```

output:

Dimension	Avg Distance	Accuracy
2	2.12	0.97
5	3.85	0.96
10	5.62	0.94
20	8.34	0.91
50	13.56	0.87

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand